Using a LED Blinking Program with FreeRTOS

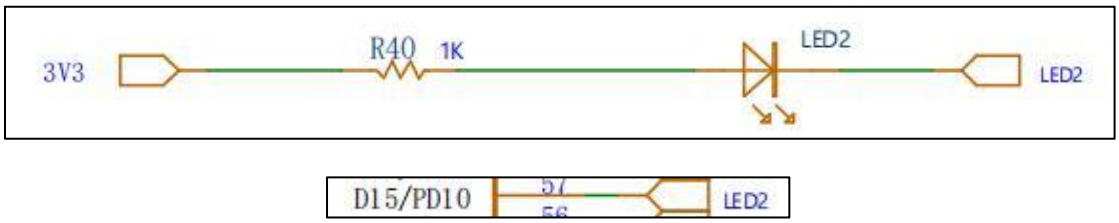
1. Program Logic

This experiment utilizes the FreeRTOS operating system to create a user task. Within this task, the PE10 pin's high and low levels are manipulated to control the blinking of an LED.

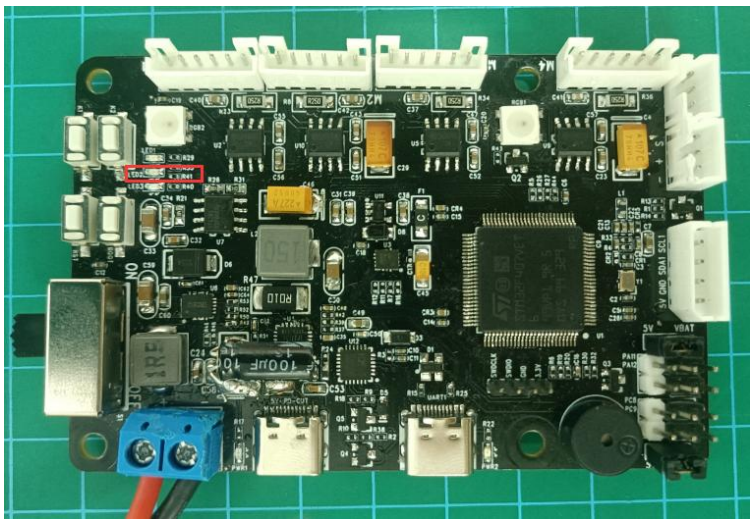
2. Hardware Connection

The LED is connected to pin PE10, where it is off when the pin is at a high level and on when it is at a low level.

Below is the circuit diagram:



The LED on the controller is as below:



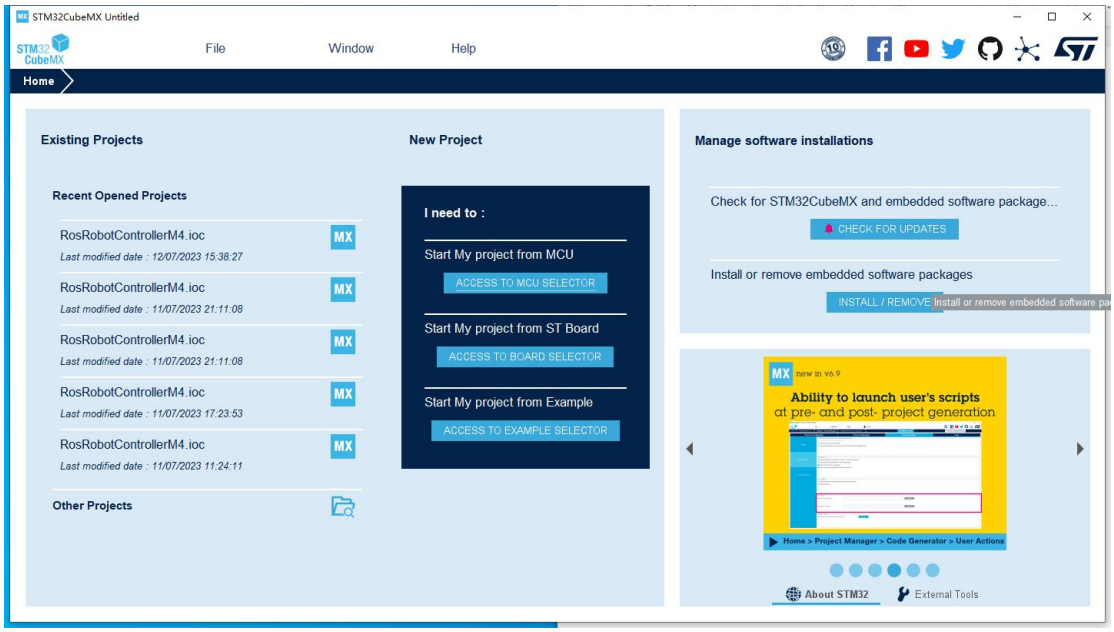
3. Program Download

The example program for this section is named "Hiwonder_FreeRTOS_LED".

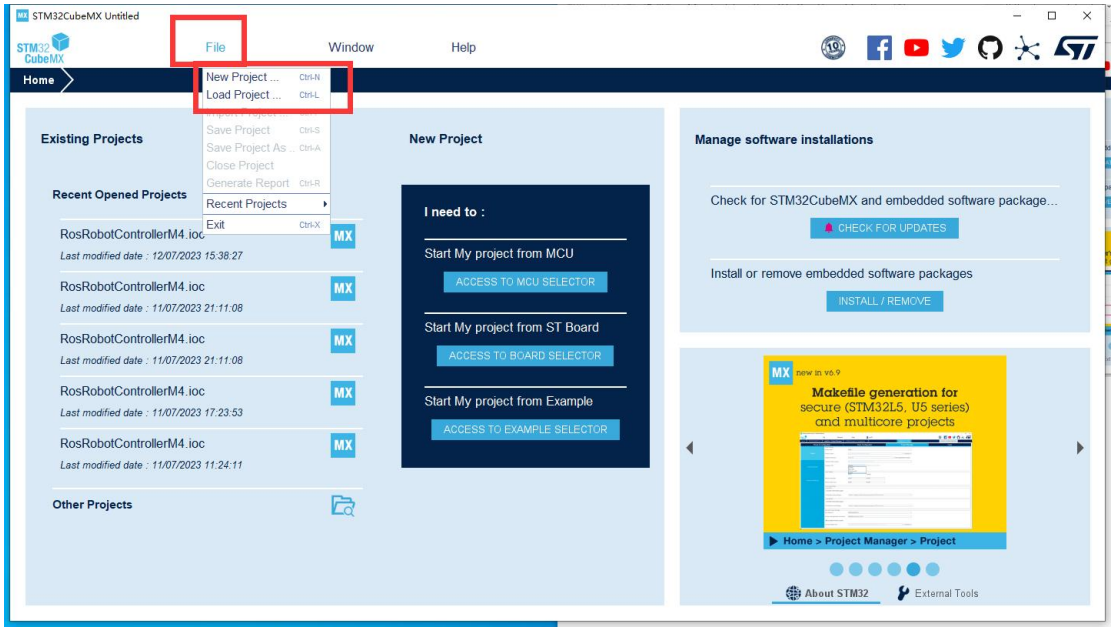
4. Project Creation and Program Writing

4.1 Project Creation

1. After double-clicking to open the STM32CubeMX software, the following interface will appear:

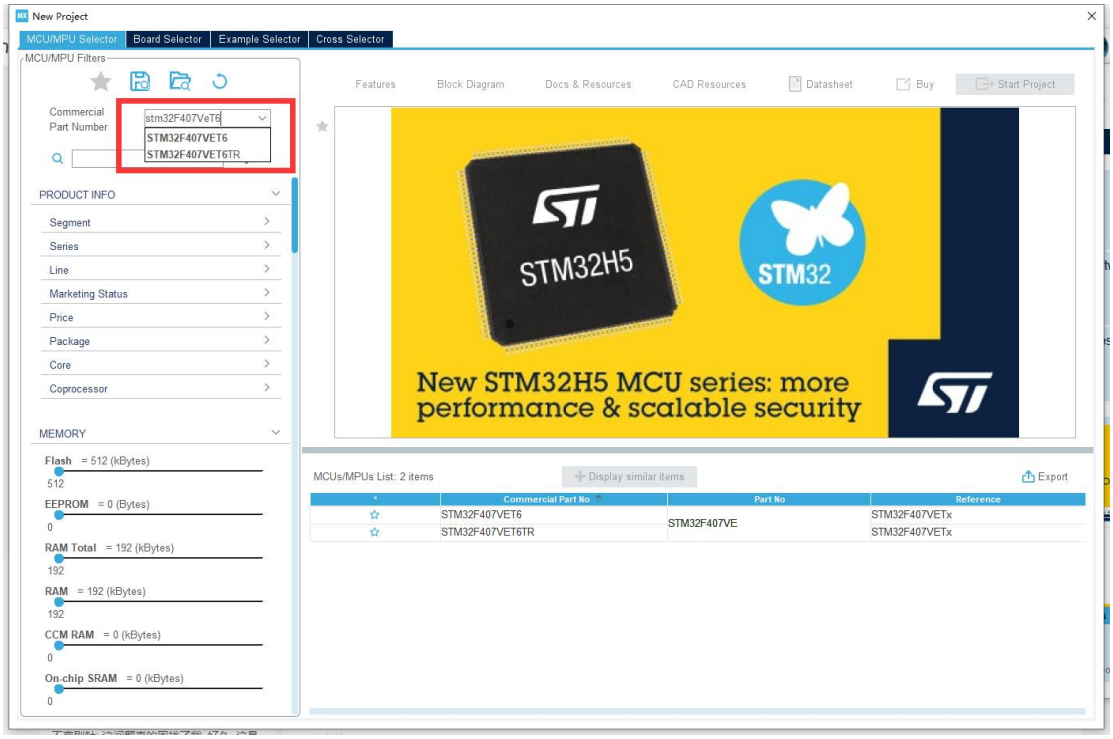


2. Click on "File". If you already have a project created by STM32CubeMX, you can click on the second option "Load Project..."; if you want to create a new project, click on the first option "New Project".

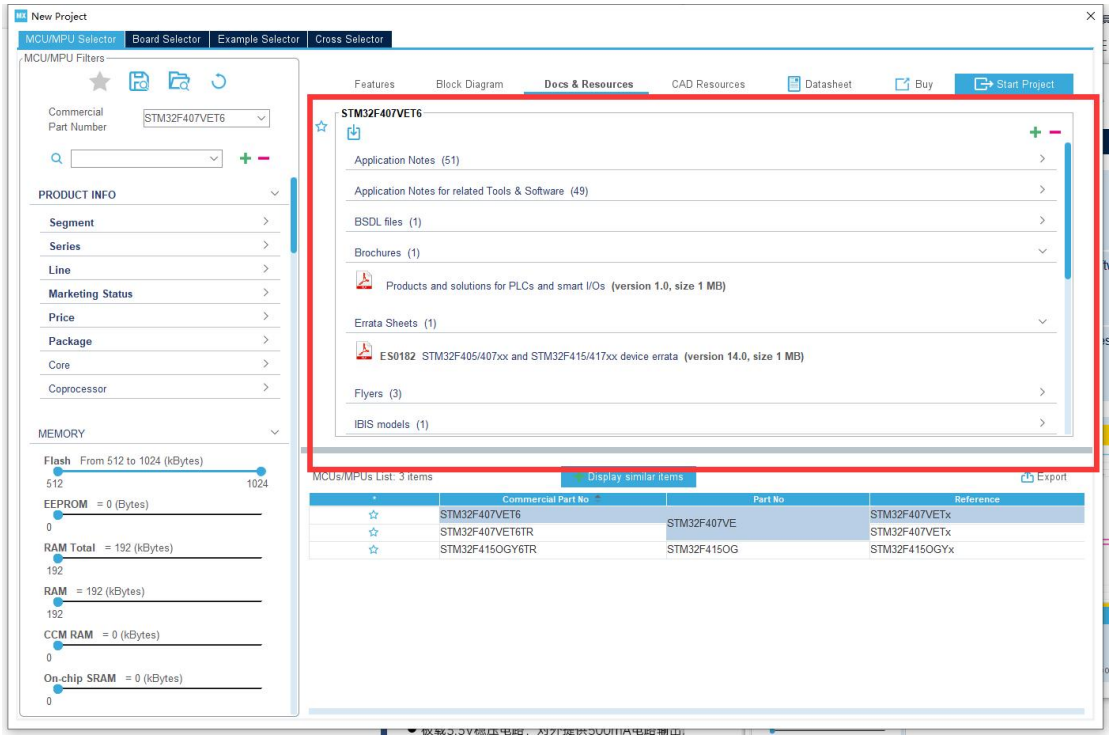


Wait for the plugins to finish loading.

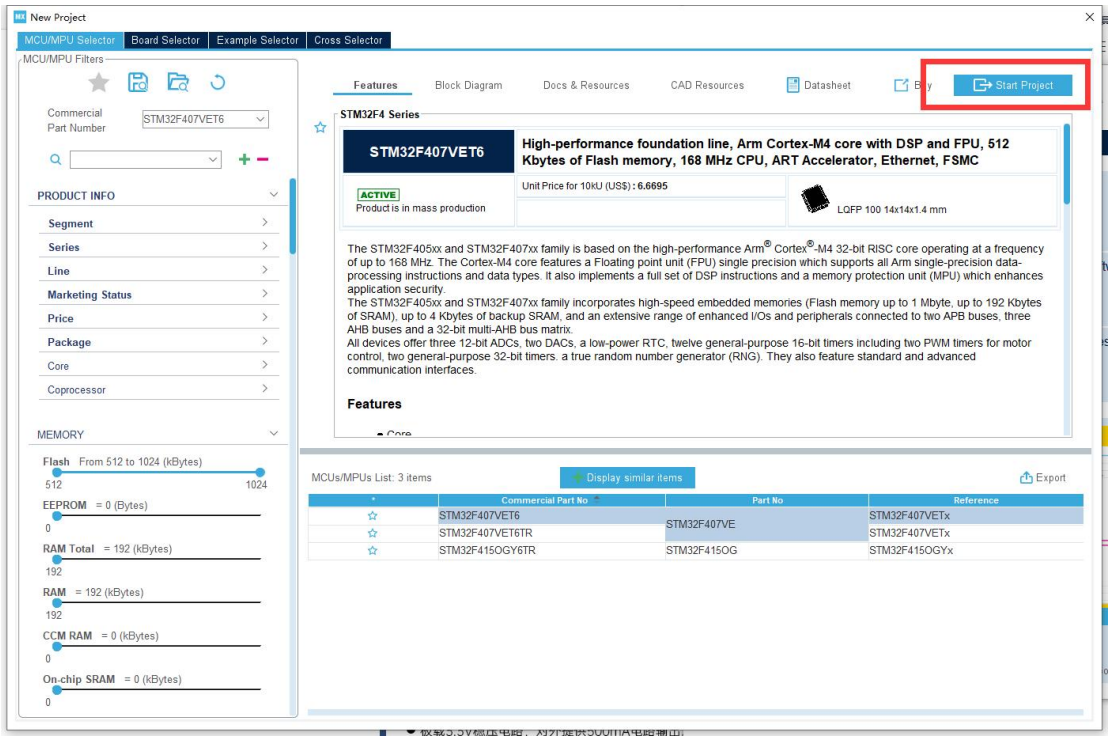
3. In the input box below, enter the selected chip model: STM32F407VET6



4. After selecting the chip, you can choose the necessary chip information in this box.



5. Then click on "Start Project" to begin creating a new project.

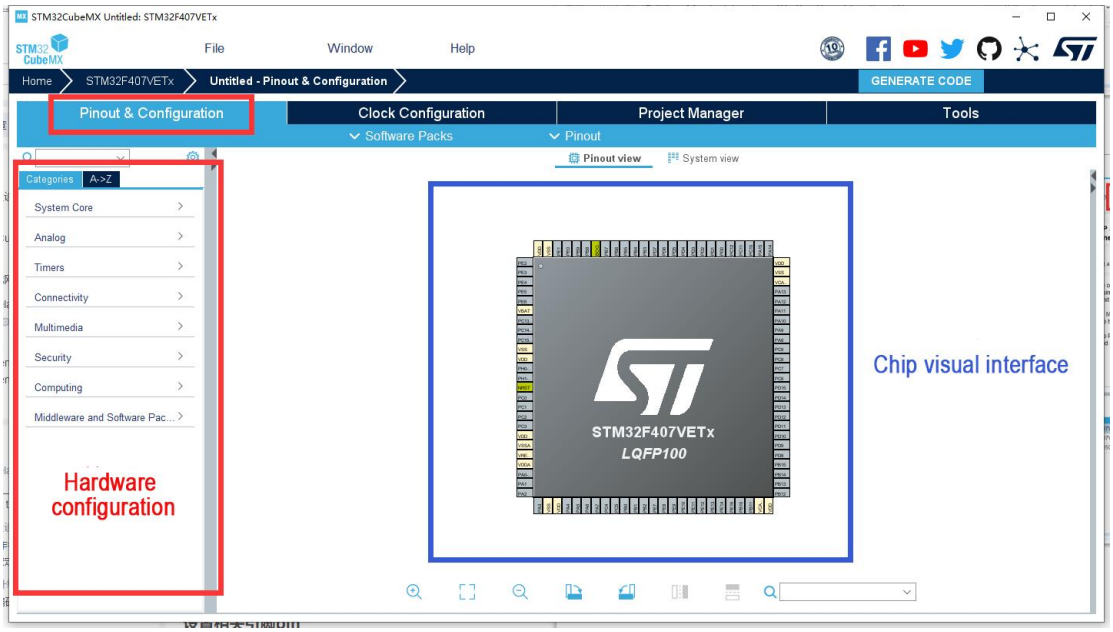


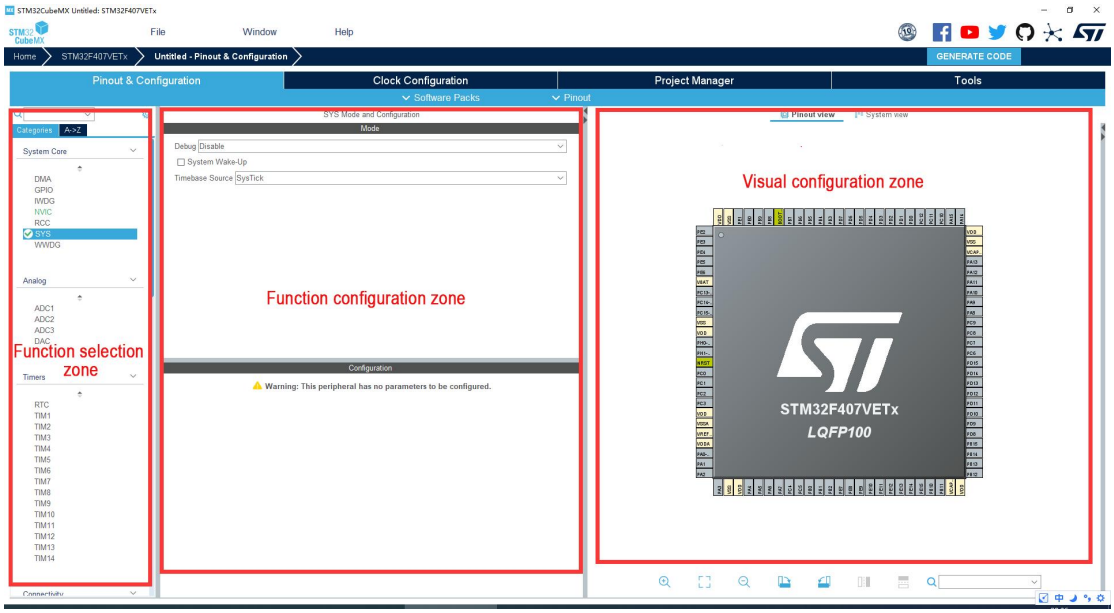
4.2 Chip Configuration & Project Creating

4.2.1 Chip Configuration Interface

(1) Pinout & Configuration :

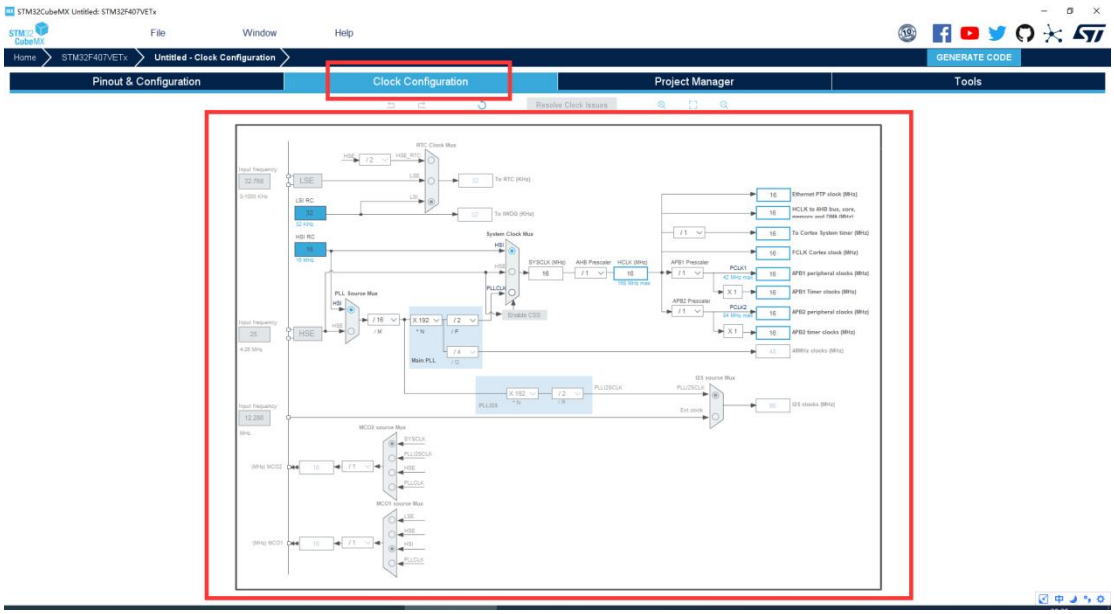
On the left are hardware configuration options, and on the right is the visual interface for chip operation, where you can configure the IO ports.





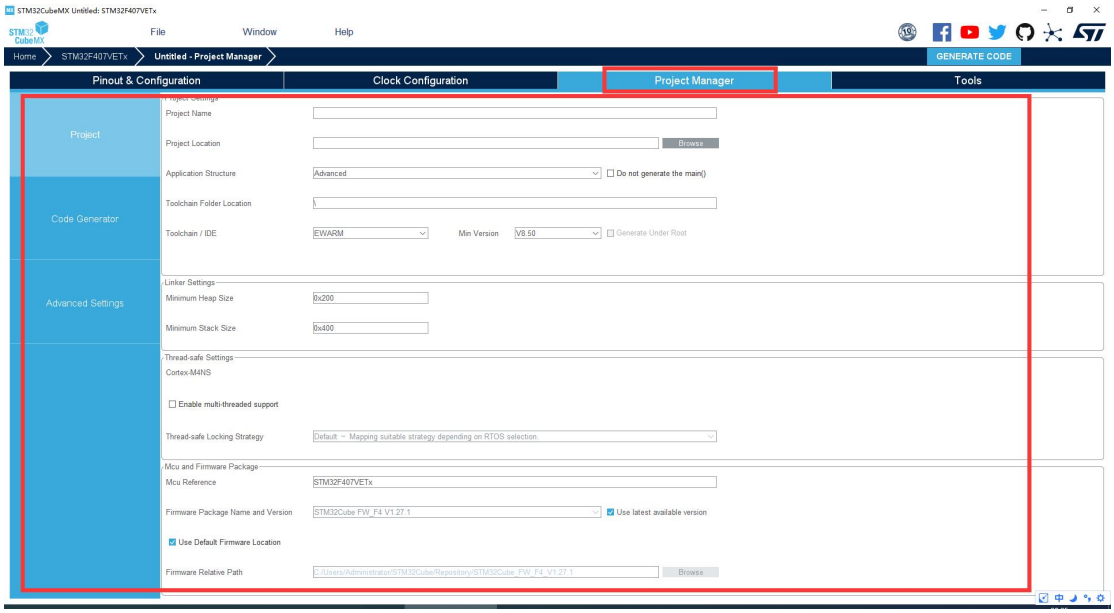
(2) Clock Configuration:

This interface is for configuring the clock of various parts of the chip.



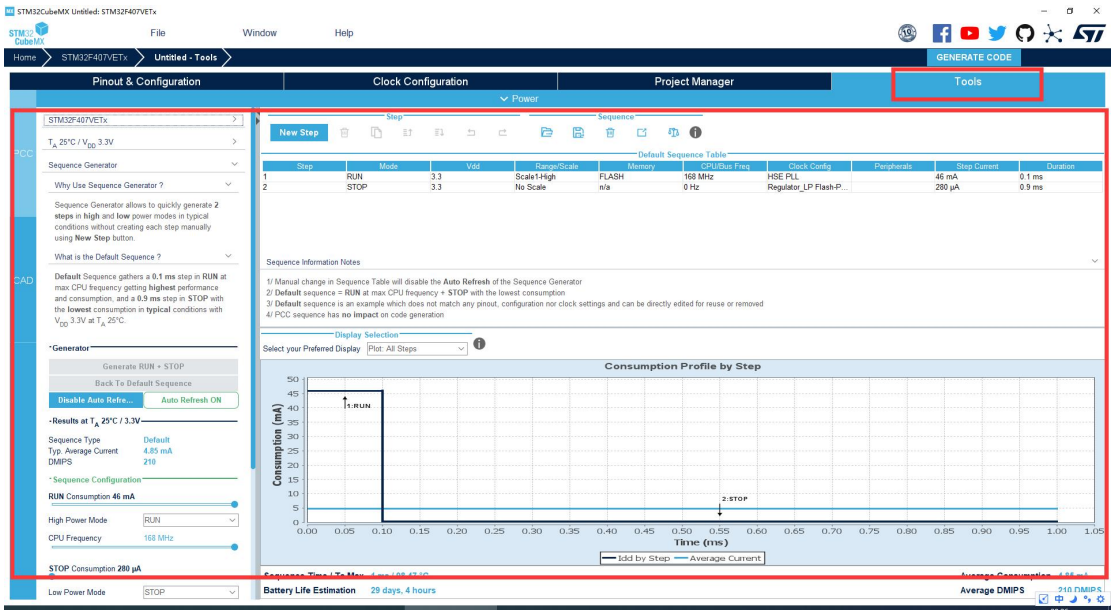
(3) Project Manager:

This interface is the project management interface, where various parameters for generating project files are configured.



(4) Tools:

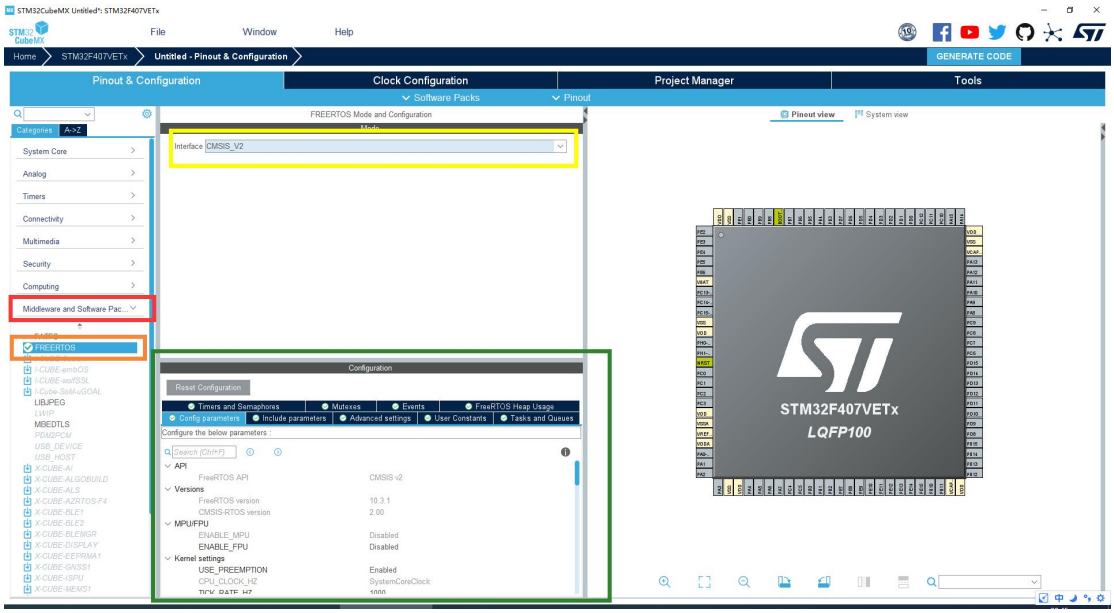
This interface is the tools interface, where these tools can be used during the development process.



4.2.2 Pinout & Configuration Interface Configuration

1) Click on "**Middle and Software Packs**", choose "**FreeRTOS**", then select the CMSIS_V2 version to use FreeRTOS in the project.

2) Configure the relevant parameters of the RTOS system in the configuration.



In the Configuration box, there are many options for creating system functionalities:

Timers and Semaphores: Create timers and semaphores.

Mutexes: Create mutexes.

Events: Create events.

FreeRTOS Heap Usage: Set up the heap for the system and tasks.

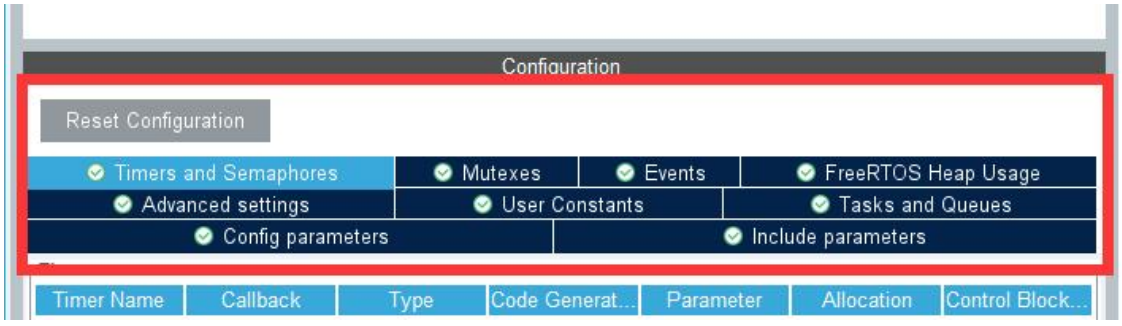
Advanced Settings: Advanced system settings.

User Constants: Create user constants.

Tasks and Queues: Create tasks and queues.

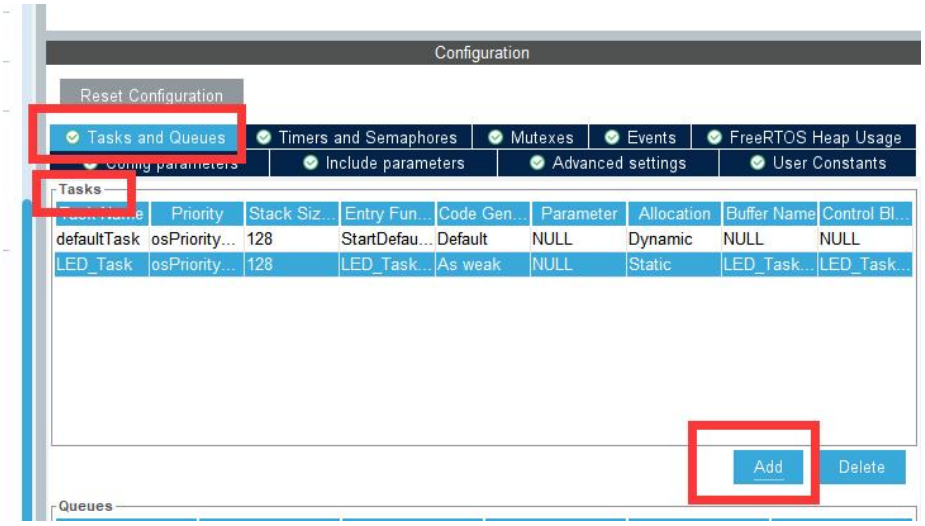
Config Parameters: Configure the system kernel.

Include Parameters: Configure the system include program.

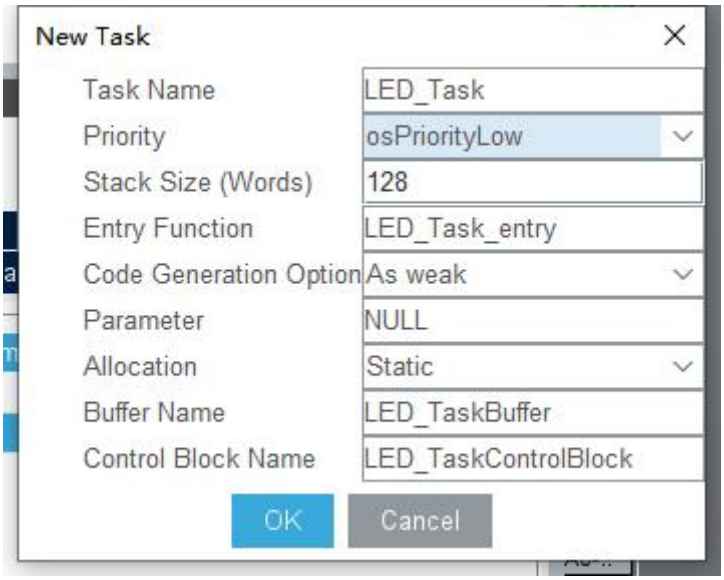


3) Create Task

Select the "Tasks and Queues" option, then click "Add" in the Task section to add a task.



The task creation interface is as below:



Task Name: The name of the task, where you can define your own task name.

Priority: The priority of the task, usually set to default.

Stack Size: The size of the task's stack, typically set to default.

Entry Function: The entry function of the task, where you write the task loop. You can name it as you wish.

Code Generation Option: Generates virtual function code for the task's entry function.

Parameter: Input parameters, usually set to NULL.

Allocation: The allocation method for the task, either dynamic or static. Here, we choose to create a static task.

Buffer Name: The name of the task's buffer, usually set to default.

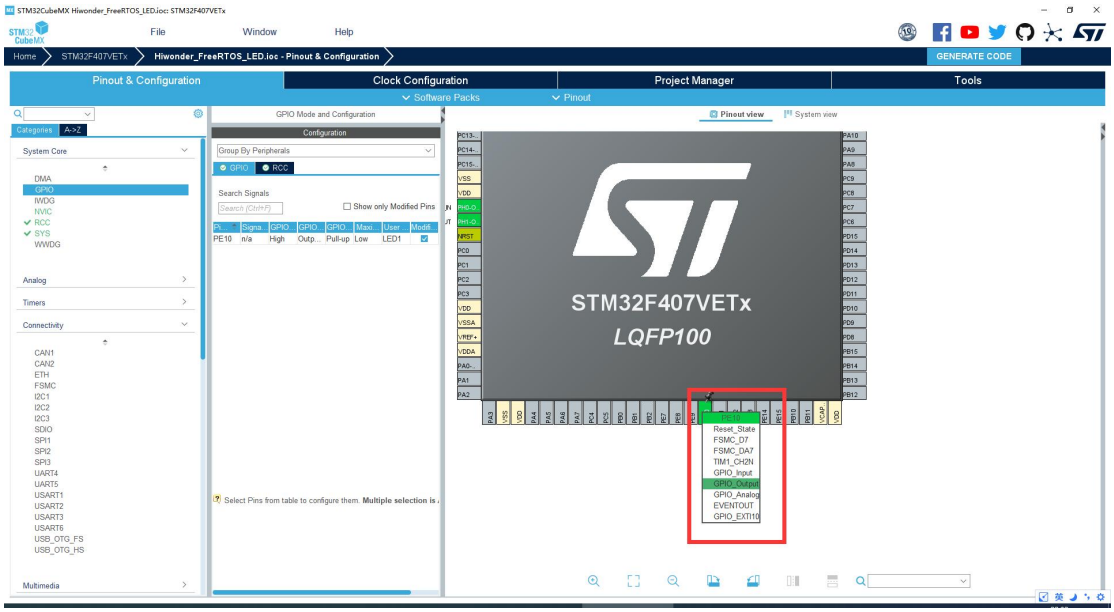
Control Block Name: The name of the task's control block, usually set to default.

After configuring, click "OK" to create the task.

4) Configure IO Port

One end of the LED on the development board is connected to 3.3V, and the other end is connected to the PE10 pin. Therefore, we need to configure the PE10 pin. When the pin is high, the LED is off (not conducting); when the pin is low, the LED is on (conducting).

Click on the PE10 pin in the visual interface and set it to GPIO_Output.



Select the "System Core" dropdown menu, click on the "GPIO" option, and configure it as follows:

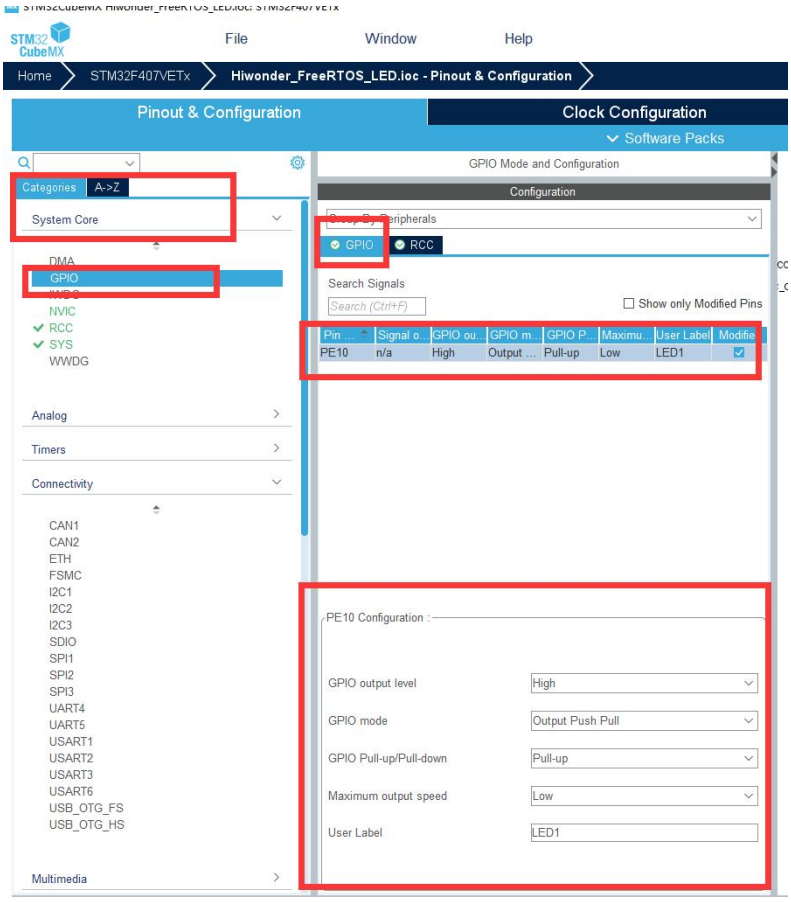
Default output: High

Output mode: Output Push Pull

Pull: Pull-up

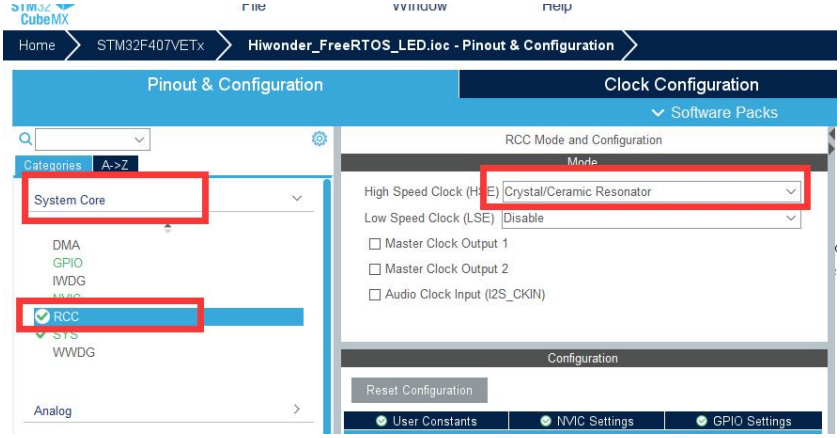
Output speed: Low

User-defined label: LED1 (you can name this as you wish, and it will be associated with the IO macro definition in the program).



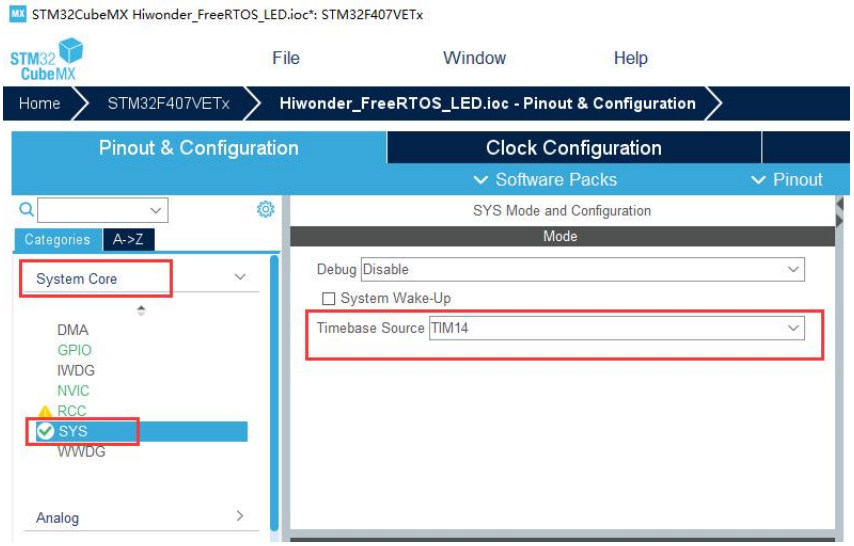
5) Configure RCC

Click on the RCC option, select External High Speed Clock (HSE) as Crystal/Ceramic Resonator (passive external oscillator).



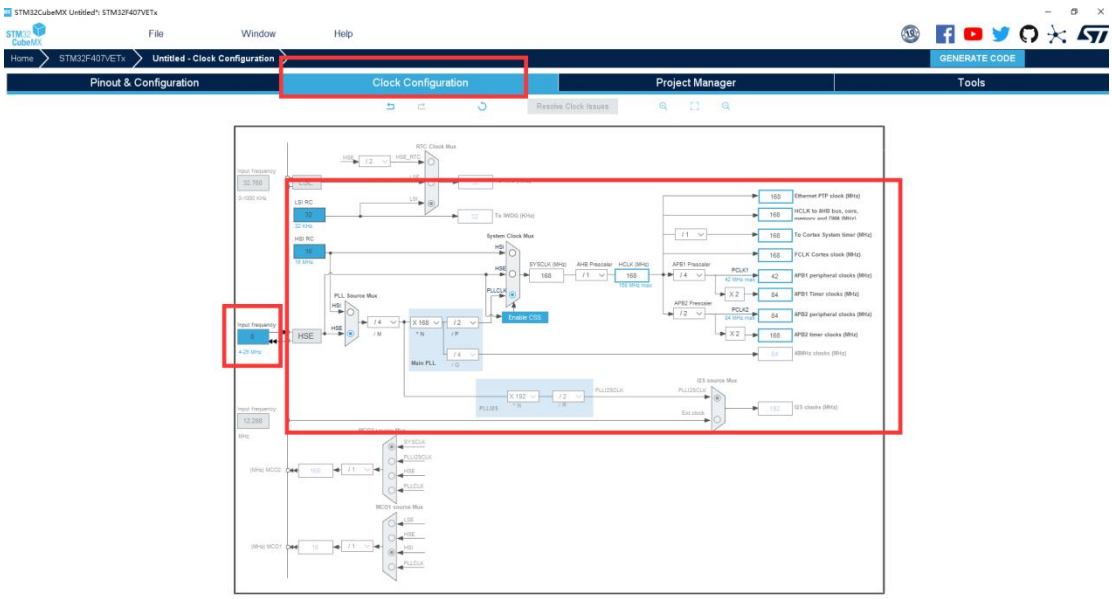
6) Configure SYS

Select TIM14 (Timer 14) as the system clock.

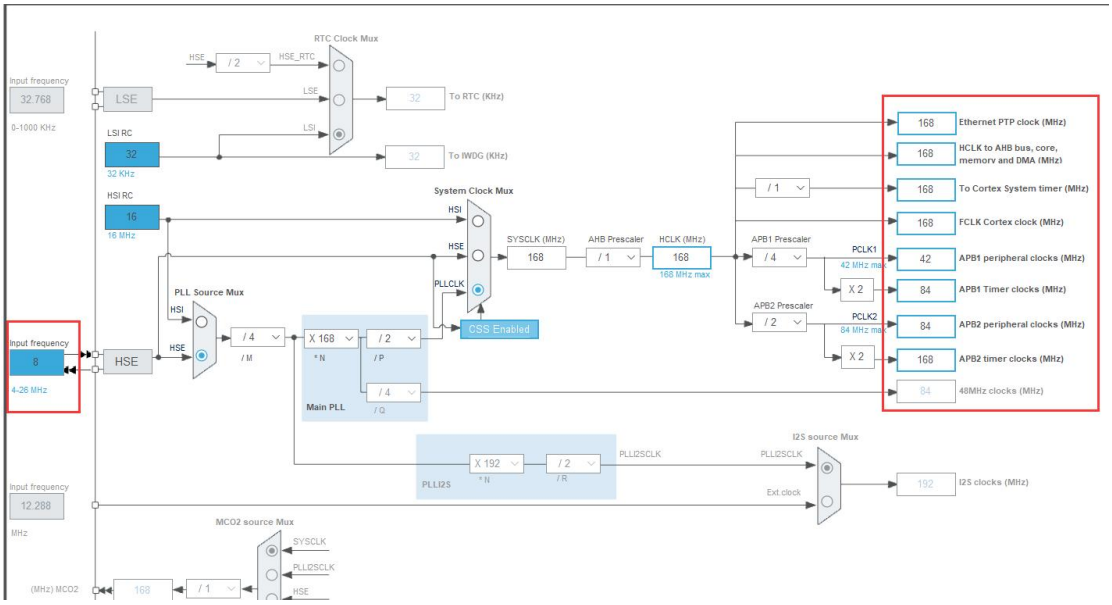


4.2.3 Clock Configuration Interface Configuration

Click on "**Clock Configuration**". The external oscillator of the development board is 8 MHz, so enter 8 in the "Input frequency" field, then configure according to the diagram below.



You only need to focus on the parameters at the end that will be used in the projects.



4.2.4 Project Manage Interface Configuration

Click on "Project Manager" to enter the project generation configuration, focusing mainly on the "Project Settings" under the "Project" tab.

Project Name: Name of the project

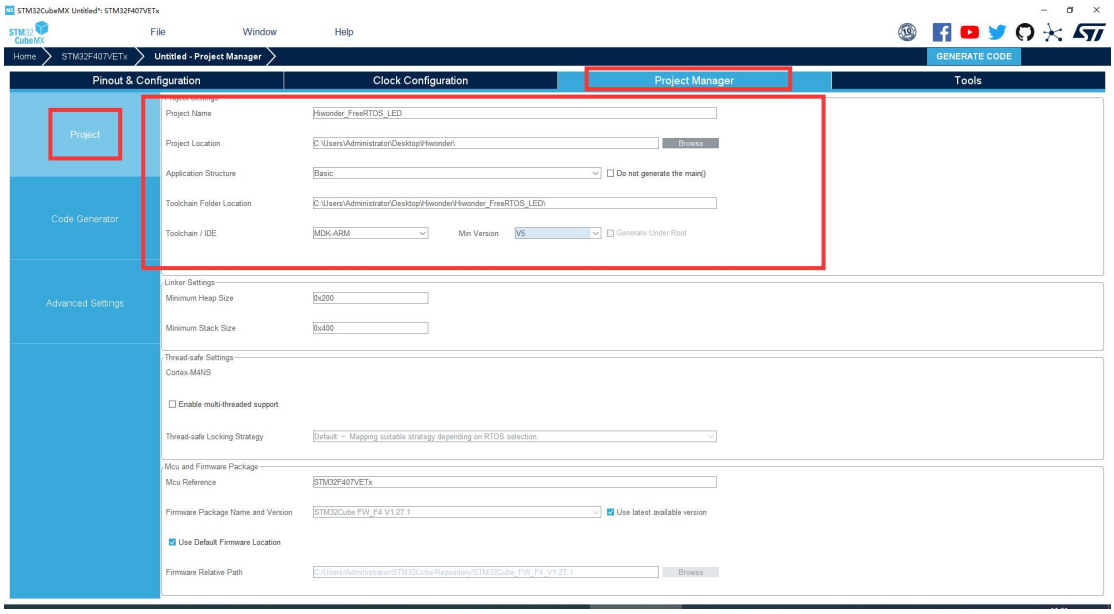
Project Location: Local path for the project

Application Structure: Choose "Basic"

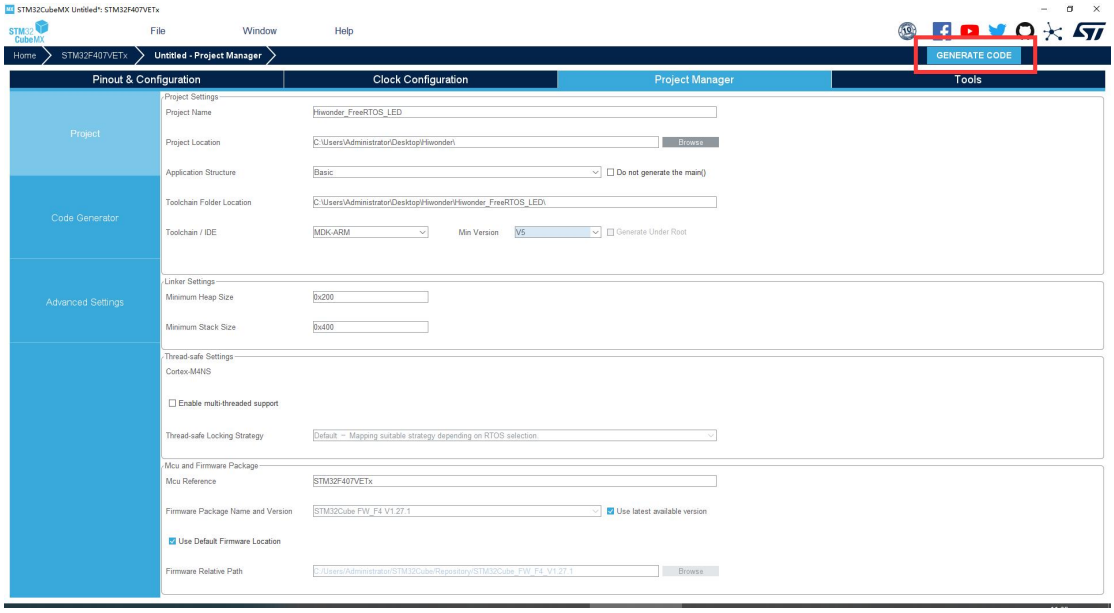
Toolchain Folder Location: Default location

Toolchain / IDE: Choose the programming and compiling tool. We are using Keil5 for programming, so select "MDK-ARM"

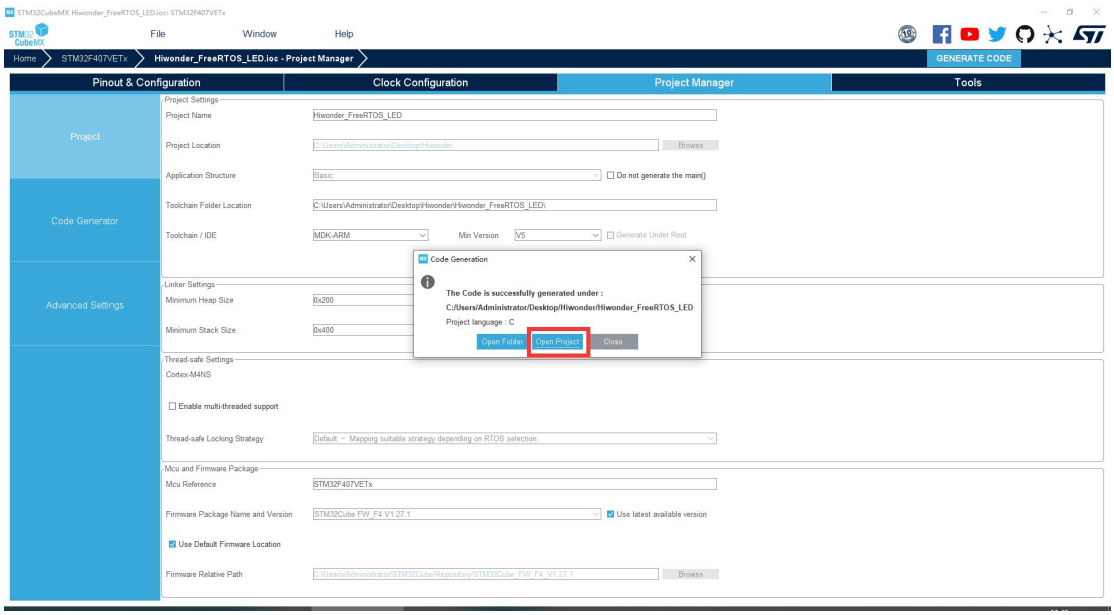
Min Version: Select the minimum version as "V5"



Click-on '**GENERATE CODE**' button at the upper right corner.

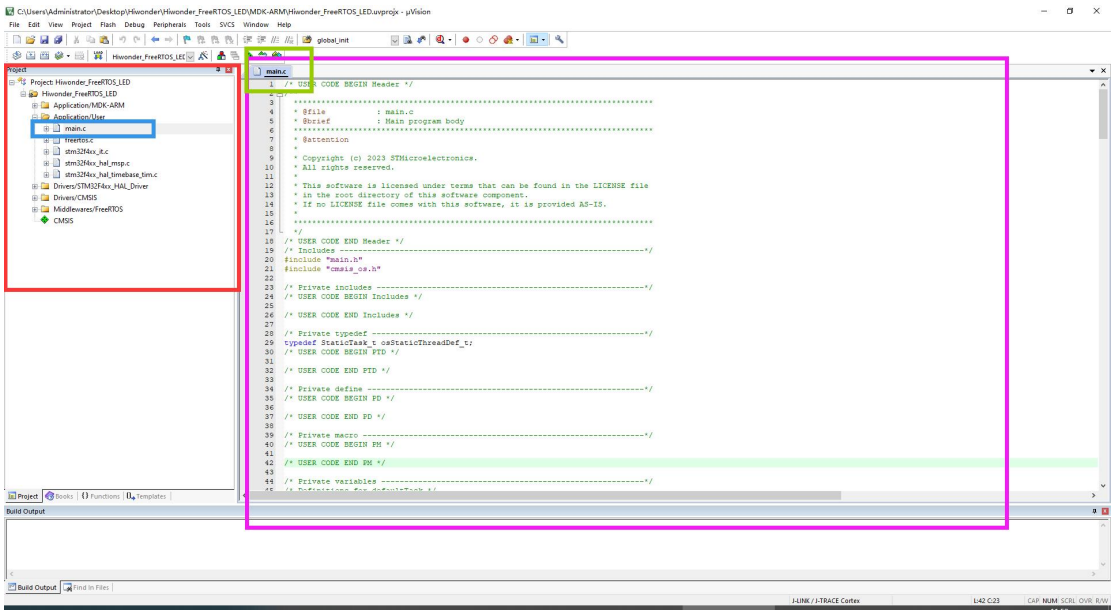


After generation, click "Open Project" to directly open the project with Keil5 software (Keil5 needs to be installed).



4.3 LED Program Editing

Open main.c file in Application / User.



For project files generated by STM32CubeMX, users' program code and comments need to be added or modified within specific ranges, as shown below:

```
/* USER CODE BEGIN xxx*/
//User comments and program code
/* USER CODE END xxx*/
```

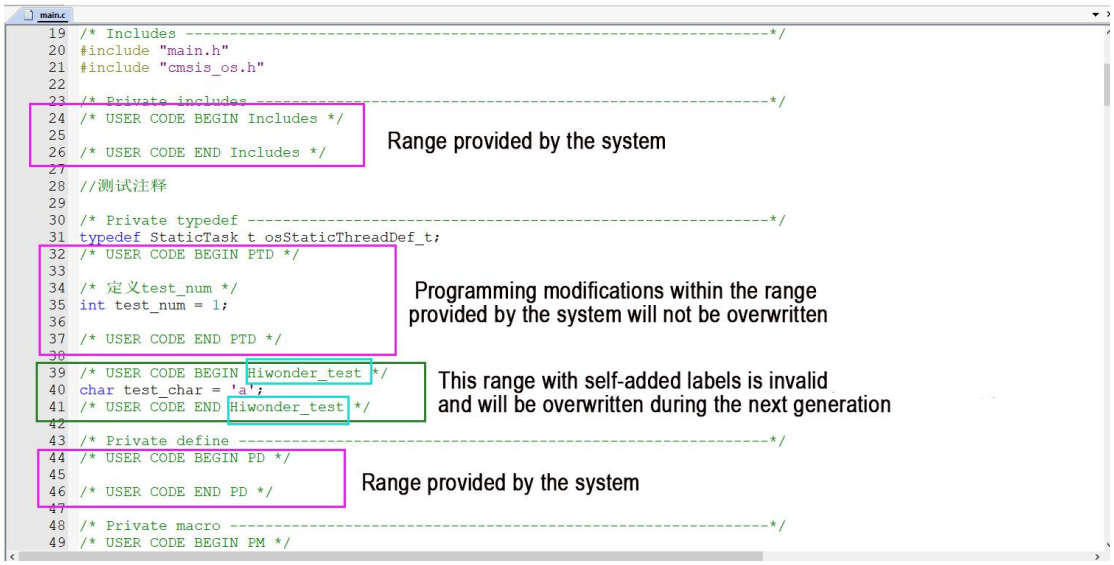
If modifications are made outside of this range, they will be overwritten during

the next configuration modification and regeneration in STM32CubeMX. Only the code and comments within this range will be preserved.

For files created and added manually, this restriction does not apply.

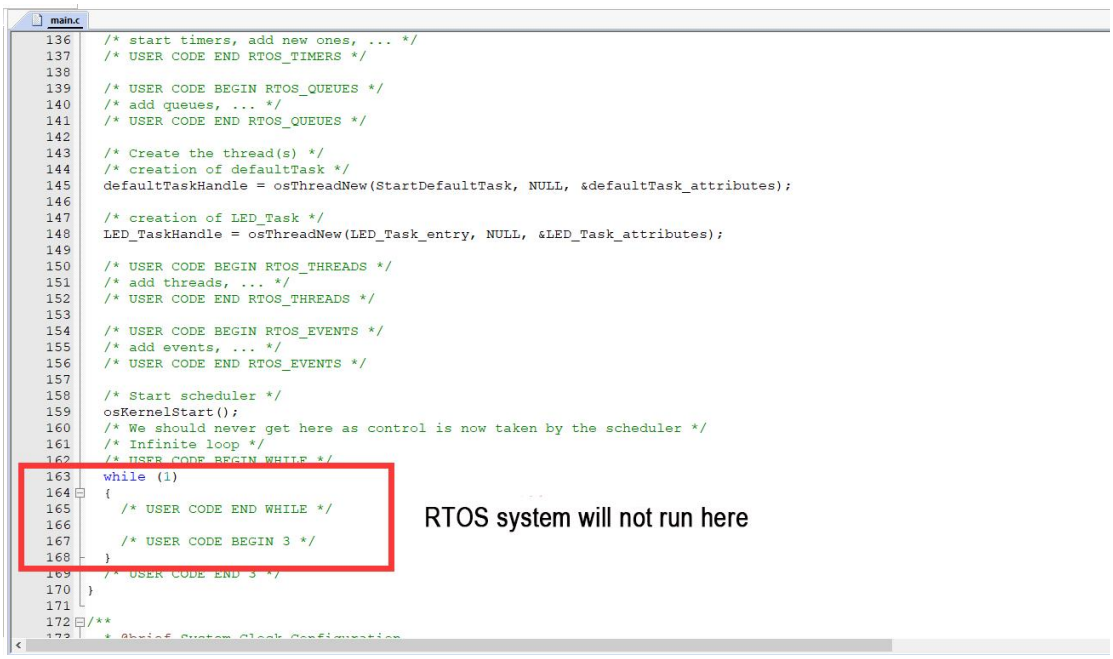
Note: If you manually add this tag, STM32CubeMX software will not recognize it and will still overwrite the code outside of this range.

```
19  /* Includes ------------------------------------------------------------------
20  #include "main.h"
21  #include "cmsis_os.h"
22
23  /* Private includes ----------------------------------------------------------
24  /* USER CODE BEGIN Includes */
25
26  /* USER CODE END Includes */
27
28  /* Private typedef -----------------------------------------------------------
29  typedef StaticTask_t osStaticThreadDef_t;
30  /* USER CODE BEGIN PTD */
31
32  /* USER CODE END PTD */
33
34  /* Private define ------------------------------------------------------------
35  /* USER CODE BEGIN PD */
36
37  /* USER CODE END PD */
38
39  /* Private macro -------------------------------------------------------------
40  /* USER CODE BEGIN PM */
41
42  /* USER CODE END PM */
43
44  /* Private variables ---------------------------------------------------------
45  /* Definitions for defaultTask */
```



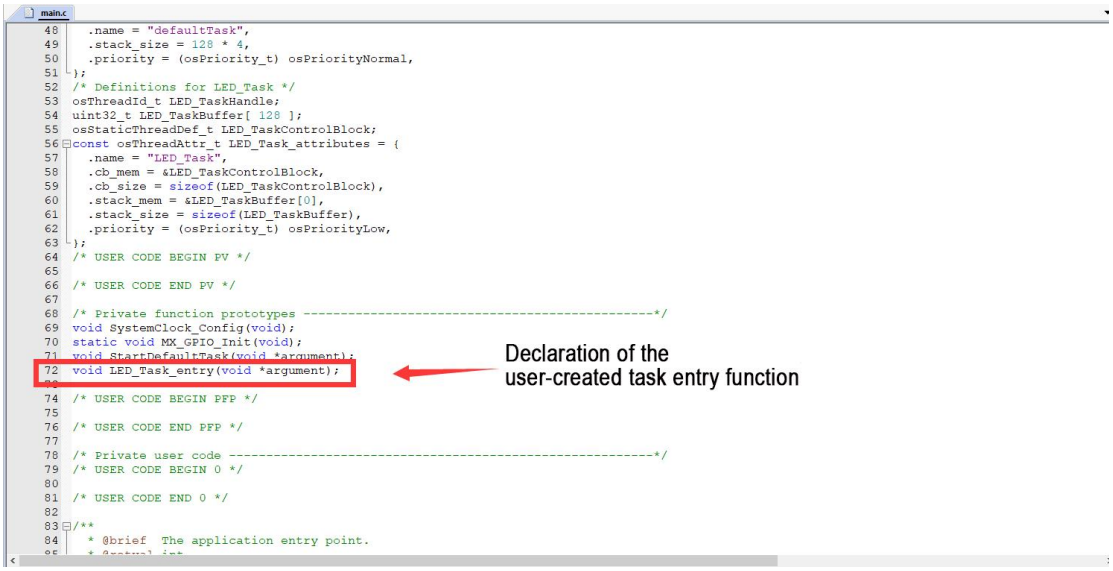
The while loop in the main() function will not be reached by the system, so LED

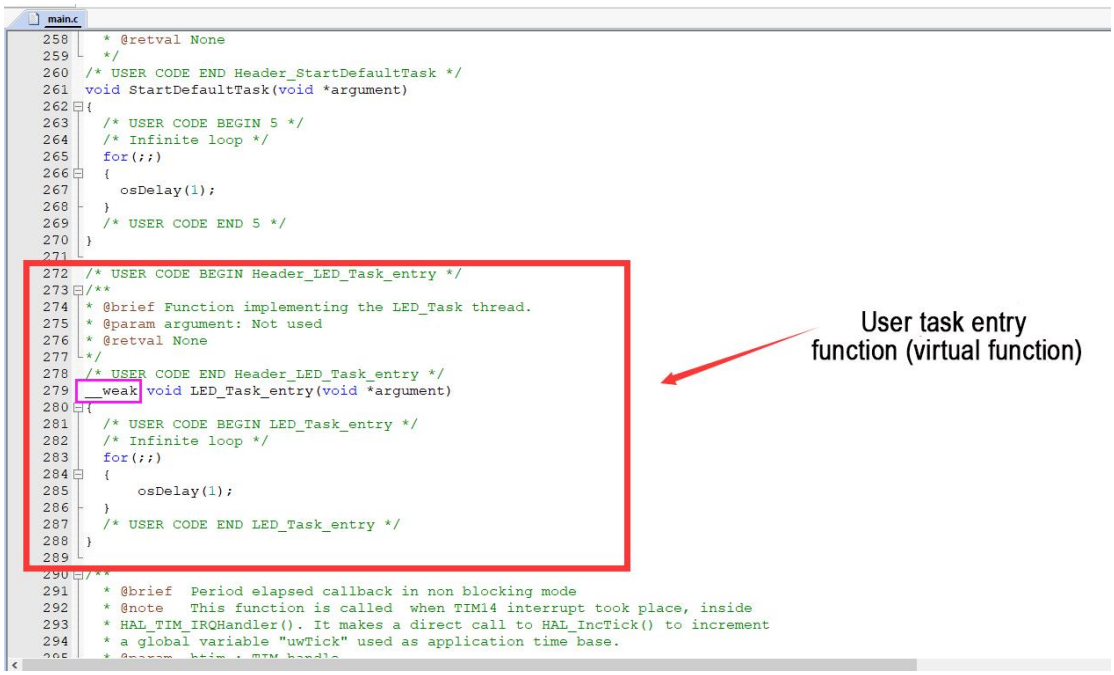
programs cannot be written here.



You need to find the LED_Task_entry() function that we defined as the task entry function when configuring the FreeRTOS system (section 2.3). Write your LED blinking logic inside this task function.

In the main.c file, you can find the declaration and virtual function definition of the LED_Task_entry() function.





Our LED blinking program can be placed inside this function. Make sure to write it within the range provided by the system for ease of reconfiguration later.

```
/* USER CODE BEGIN Header_LED_Task_entry */
/**
* @brief Function implementing the LED_Task thread.
* @param argument: Not used
* @retval None
*/
/* USER CODE END Header_LED_Task_entry */
__weak void LED_Task_entry(void *argument)
{
  /* USER CODE BEGIN LED_Task_entry */

  /* Infinite loop */
  for(;;)
  {
      HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
      osDelay(500);
      HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
      osDelay(500);
  }

  /* USER CODE END LED_Task_entry */
}
```

Program logic:

for(; ;){

Set the LED pin to high level;

Delay for 500ms;

Set the LED pin to low level;

```
    Delay for 500ms;
}
```

After compilation, once there are no errors indicated below, you can proceed to download and burn the program.